实验报告

25300440003 徐培森

0. 前言

0.1 外部依赖与环境配置说明

1. JDK版本：Openjdk version "11.0.30"
2. IDE环境：InteliJ idea
3. 第三方库：Lanterna 3.1.1

1. 项目结构

1.1 结构层次

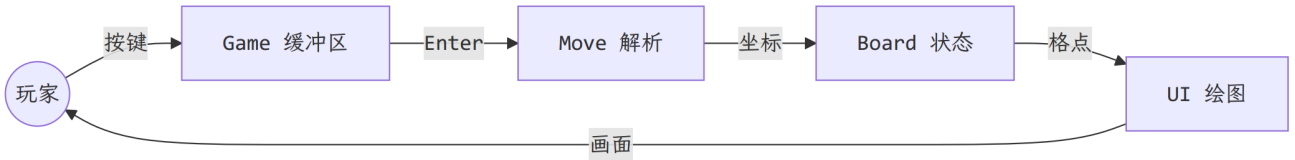
```
com.othello
├── Main.java                    # 程序入口
├── model/                       # 数据模型层
│   ├── Board.java                 # 棋盘模型
│   ├── Piece.java                 # 棋子枚举
│   ├── Player.java                # 玩家模型
│   └── Move.java                  # 移动记录
├── logic/                       # 业务逻辑层
│   └── Game.java                  # 游戏控制器
└── ui/                          # 用户界面层
    └── TerminalUI.java            # 终端界面
```

1.2 设计模式

本次lab使用了MVC设计模式，说明如下

- 模型层（Model）：Board、Piece、Player、Move类
- 视图层（View）：TerminalUI类
- 控制层（Controller）：Game类

1.3 数据流向



2. 类设计详细说明

2.1 Main.java

2.1.1 主要功能

`Main.java` 是整个黑白棋游戏的入口类，其核心职责如下：

1. **实例初始化**：创建并管理 `com.othello.logic.Game` 对象的实例。
2. **异常管理**：捕获并记录游戏运行期间未处理的所有严重错误。

2.1.2 关键代码及设计思路

1. **try-with-resources 资源管理**

```
try (Game game = new Game()) {
    game.start();
} catch (Exception e) {
    e.printStackTrace();
}
```

- **自动关闭逻辑**：由于 `Game` 类实现了 `AutoCloseable` 接口，使用这种 `try(...)` 语法后，Java 编译器会在 `try` 块结束时自动调用 `game.close()` 方法。

2.1.3 设计特点

1. **安全性高**：
   不再依赖手动调用 `ui.stop()`。这种设计确保了即便在发生不可预知的错误时，程序依然能优雅地关闭终端，恢复系统控制台的正常显示。
2. **代码精简**：
   `Main` 类不包含任何有关游戏的操作，将所有参数设置交由 `Game` 内部或初始化流程处理。

2.2 Game.java

2.2.1 主要功能

- **管理游戏周期**：控制从"棋盘初始化"到"游戏循环"再到"资源释放"的全过程。
- **输入处理**：利用缓冲区，在字符终端实现实时输入与退格编辑功能。
- **逻辑协调**：调用模型层验证坐标合法性、执行落子规则，并实时更新回合状态。
- **状态反馈系统**：维护游戏胜负判定、错误信息提示以及 UI 高亮状态（`isError` 标记）。

2.2.2 关键代码及设计思路

1. 主循环

```
while (true) {
    checkGameOver(); // 判定胜负
    ui.drawGame(board, currentPlayer, inputBuffer.toString(), statusMessage, gameOver, isError); // 渲染界面

    KeyStroke ks = ui.readInput();
    if (ks.getKeyType() == KeyType.Enter) {
        String cmd = inputBuffer.toString().trim();
        if (cmd.equalsIgnoreCase("quit")) break; // 退出逻辑
        if (!gameOver) handleMove(cmd); // 处理落子
        inputBuffer.setLength(0); // 清空缓冲区
    }
```

```
        else if (ks.getKeyType() == KeyType.Backspace && inputBuffer.length() > 0) {
            inputBuffer.deleteCharAt(inputBuffer.length() - 1); // 退格删除
        }
        else if (ks.getKeyType() == KeyType.Character && inputBuffer.length() < 10) {
            inputBuffer.append(ks.getCharacter()); // 字符追加
        }
    }
```

- 实时反馈：循环一开始就调用 `drawGame`，配合 `inputBuffer` 的实时追加，使用户每输入一个字母都能在屏幕上即时看到预览。
- 长度约束：`inputBuffer.length() < 10` 限制了非法超长输入，从物理层面保证了 UI 界面不会因超长字符串而产生错误。
- 双模式兼容：通过 `gameOver` 使用同一套输入逻辑，在游戏结束后依然允许用户输入 "quit" 退出，保持了交互的一致性。

2. 处理落子

```
    private void handleMove(String cmd) {
        Move move = Move.fromString(cmd); // 1. 格式转换层
        if (move == null) {
            statusMessage = "格式错误! 请输入如 A1 的坐标。";
            isError = true;
            return;
        }

        int r = move.row(), c = move.col();
        if (board.isOutOfBounds(r, c)) { // 2. 物理边界校验
            statusMessage = "坐标超出范围! ";
            isError = true;
            return;
        }

        if (board.isOccupied(r, c)) { // 3. 占用逻辑校验
            statusMessage = "该位置已有棋子! ";
            isError = true;
            return;
        }

        if (board.place(r, c, currentTurn)) { // 4. 业务规则执行
            statusMessage = "";
            isError = false;
            currentTurn = (currentTurn == Piece.BLACK) ? Piece.WHITE : Piece.BLACK;
        }
    }
```

- 严谨的检查机制：
  - 采用多个 `if-return`，将错误情况优先排除，保证程序正确运行而且提高了代码可读性。
  - 只有当 `board.place` 返回 `true` 时(合法落子)，才会切换 `currentTurn`。这种设计防止了无效落子导致的回合轮数错误。

2.2.3 设计特点

1. **安全性高**：通过实现 `AutoCloseable` 接口，将终端环境的清理逻辑封装在 `close()` 方法中，能确保即便程序崩溃，用户终端也正常退出。
2. **高内聚低耦合**：`Game` 类不直接修改棋盘二维数组，而是通过 `Board` 提供的方法进行交互，并向 `TerminalUI` 传递渲染所需的基础数据。
3. **交互反馈友好**：引入 `isError` 布尔标记。当验证失败时，该标记驱动 UI 层将 `statusMessage` 以红色或其他醒目方式呈现，弥补了命令行界面交互直观性不足的缺点。
4. **容错机制健壮**：在设置阶段（`handleSetup`）和落子阶段（`handleMove`）均使用了完备错误捕获与提示，确保了程序在面对非法输入时不会崩溃。

2.3 Board.java

2.3.1 主要功能

- **保存数据**：使用二维数组存储棋盘格点状态（黑、白、空）。
- **初始化**：支持动态棋盘尺寸，并自动设置黑白棋开局的十字交叉阵型。
- **合法性校验**：判断坐标是否越界、目标位置是否已被占用。
- **状态统计**：实时统计黑白双方棋子数量，并判定棋盘是否已满。

2.3.2 关键代码及设计思路

1. 开局初始化

```
public Board(int size) {
    this.size = size;
    this.grid = new Piece[size][size];

    // 1. 全局初始化为空
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++) grid[i][j] = Piece.EMPTY;

    // 2. 核心十字阵型计算
    int mid = size / 2;
    grid[mid - 1][mid - 1] = Piece.WHITE; // 左上白
    grid[mid][mid] = Piece.WHITE;         // 右下白
    grid[mid - 1][mid] = Piece.BLACK;     // 右上黑
    grid[mid][mid - 1] = Piece.BLACK;     // 左下黑
}
```

- **动态适配**：
  - 通过 `size / 2` 计算中点，确保无论棋盘是 4x4 还是 16x16，初始棋子始终位于棋盘几何中心。
  - 严格遵循黑白棋标准，将中间四个格子设为交叉排列，保证后续所有合法移动。

2. 状态原子判定与放置

```
public boolean place(int r, int c, Piece piece) {
    // 1. 组合验证：防止越界导致数组崩溃，防止覆盖已有棋子
    if (isOutOfBounds(r, c) || isOccupied(r, c)) return false;

    // 2. 状态更新
```

```
        grid[r][c] = piece;
        return true;
    }

    public boolean isFull() {
        // 遍历优化：一旦发现空位立即返回，提高终局检测效率
        for (int i = 0; i < size; i++)
            for (int j = 0; j < size; j++)
                if (grid[i][j] == Piece.EMPTY) return false;
        return true;
    }
```

- **安全性高**：`place` 方法拦截非法操作。
- **短路**：`isFull` 方法采用快速返回逻辑，在棋盘非满状态下能以高效率完成判定，支撑 `Game` 循环的高频调用。

2.3.3 设计特点

1. **高内聚的数据管理**：棋盘的大小、格点状态、统计功能全部封装在类内部。外部类无需了解二维数组的具体实现，只需通过公开接口交互。
2. **职责单一**：`Board` 只负责"棋子能不能放下"和"现在棋盘是什么样"的物理逻辑，而不参与解析用户输入等表现层逻辑
3. **支持灵活扩展**：棋盘边长通过输入传入，而非硬编码，为后续开发不同难度或尺寸的关卡留下了空间。
4. **健壮性**：所有的坐标操作前均强制调用 `isOutOfBounds`，极大地降低了运行时产生数组溢出的风险。

2.4 Move.java

2.4.1 主要功能

- **输入解析与清洗**：处理用户原始输入的字符串，剔除多余空格并统一大小写。
- **坐标系转换**：将字母映射为列索引，将数字映射为行索引。
- **数据封装**：作为一个不可变的对象，安全地在 `Game` 与 `Board` 之间传递落子位置。
- **验证合法性**：初步过滤长度不足或非标准格式的输入，确保后续逻辑处理的对象是有效的坐标点。

2.4.2 关键代码及设计思路

1. 输入解析

```
    public static Move fromString(String input) {
        // 1. 基础边界防护
        if (input == null || input.length() < 2) return null;

        // 2. 输入标准化：去除空格并转为小写，增强容错性
        String clean = input.trim().toLowerCase();

        try {
            // 3. 列转换：利用 ASCII 字符偏移计算（'a' 为 0，'b' 为 1...）
            int col = clean.charAt(0) - 'a';
```

```
11
12          // 4. 行转换：截取字母后的部分并转为整数，同时适配数组从 0 开始的特性
13          int row = Integer.parseInt(clean.substring(1)) - 1;
14
15          return new Move(row, col);
16      } catch (Exception e) {
17          // 5. 异常捕获：若 substring 无法转为数字（如 "Ax"），返回 null 标识非法格式
18          return null;
19      }
20  }
```

- **字符算术**：通过 `clean.charAt(0) - 'a'` 直接计算列索引，高效且简洁
- **动态行解析**：使用 `substring(1)` 而非固定位置 `charAt(1)`，使代码能够支持双位数的行（如"A10"、"A16"），增强了对大棋盘的适配能力。
- **安全性**：整个解析过程被包围在 `try-catch` 中，能优雅地处理各种错误输入，而不会导致系统崩溃。

2.4.3 设计特点

1. **不可变性**：`row` 和 `col` 字段被声明为 `private final`。这意味着一旦 `Move` 对象被创建，其坐标就不可能被修改，保证了在多层逻辑传递过程中的数据安全性。
2. **工厂模式**：通过 `static` 静态方法 `fromString` 创建实例，隐藏了内部构造细节，并提供了一个清晰的"字符串转对象"的方法。
3. **职责单一**：该类仅负责"坐标的转换"，它不校验坐标是否越界（由 `Board` 负责），这种设计遵循了单一职责原则，使得代码模块化程度更高。
4. **简洁**：作为一个纯粹的逻辑载体，它没有复杂的依赖，极大地降低了系统内部各组件之间的耦合度。

2.5 Piece.java

2.5.1 主要功能

- **状态定义**：明确定义格点的三种合法状态：空（EMPTY）、黑棋（BLACK）、白棋（WHITE）。
- **实现视觉化**：为每种状态绑定对应的字符（如实心圆、空心圆），以便在字符界面进行高辨识度的渲染。
- **语义化代码**：通过枚举类型提升其他代码的可读性与安全性。

2.5.2 关键代码及设计思路

1. 枚举常量与符号定义

```
1   public enum Piece {
2       // 1. 逻辑状态与视觉符号的映射
3       EMPTY(" · "),   // 空位：使用居中点表示
4       BLACK(" ● "),
5       WHITE(" ● ");
6
7       private final String display;
8
```

```
9       // 2. 内部构造函数，确保显示符号的不可变性
10      Piece(String display) {
11          this.display = display;
12      }
13
14      public String getDisplay() {
15          return display;
16      }
17  }
```

- **填充设计**：每个符号两侧均带有空格（如 `" · "`），这种设计是为了在终端中模拟出间距感，防止棋子排列过于拥挤。
- **统一封装**：通过 `getDisplay()` 方法显示文本，使得 `TerminalUI` 只需要调用此接口即可获取渲染内容，无需关心不同棋子具体对应的字符细节。

2.5.3 设计特点

1. **类型安全**：使用枚举取代整型（int）或字符串，确保在 `Board.place()` 等方法传递参数时，编译器能强制拦截除这三种状态以外的非法数据，消除了运行时状态异常风险。
2. **高内聚**：将棋子模型在模型层中定义。如果未来需要将字符改为其他样式，只需修改 `Piece` 枚举中的字符串，而无需改动 `TerminalUI` 的渲染部分代码。
3. **简洁**：该类轻量，仅包含必要的状态描述，符合"单一职责原则"。

2.6 TerminalUI.java

2.6.1 主要功能

- **动态棋盘绘制**：根据 `Board` 的尺寸，自动计算并渲染带有坐标标签（A-H，1-8）的彩色棋盘格。
- **交互引导逻辑**：分阶段展示"设置界面"与"对战界面"，并提供实时的输入缓冲区预览。
- **视觉反馈系统**：根据游戏状态（是否报错、是否结束）动态调整文本颜色和提示符号，增强操作引导。

2.6.2 关键代码及设计思路

1. 自适应棋盘渲染

```
1   for (int r = 0; r < size; r++) {
2       // 1. 绘制行索引 (1, 2, 3...)
3       graphics.putString(OFFSET_X - 3, OFFSET_Y + (r * CELL_HEIGHT) + 1, String.valueOf(r
    + 1));
4
5       for (int c = 0; c < size; c++) {
6           // 2. 棋盘背景着色：利用 (r+c) 的奇偶性实现深灰与深蓝交替的“国际象棋盘”效果
7           graphics.setBackgroundColor((r + c) % 2 == 0 ?
    TextColor.Factory.fromString("#333333") : TextColor.Factory.fromString("#0000AA"));
8
9           // 3. 物理格点绘制：在终端坐标系中计算每个 Cell 的起始位置 (sx, sy)
10          int sx = OFFSET_X + (c * CELL_WIDTH), sy = OFFSET_Y + (r * CELL_HEIGHT);
11          for (int h = 0; h < CELL_HEIGHT; h++)
```

```
            for (int w = 0; w < CELL_WIDTH; w++) graphics.setCharacter(sx + w, sy + h, ' ');

        // 4. 棋子渲染：从 Board 获取逻辑状态，并根据 Piece 枚举中的 display 字符进行居中绘制
        Piece p = board.getPiece(r, c);
        if (p != Piece.EMPTY) {
            graphics.setForegroundColor(p == Piece.BLACK ? TextColor.ANSI.BLACK : TextColor.ANSI.WHITE);
            graphics.putString(sx + 2, sy + 1, p.getDisplay().trim()); // 绘制 ● 或 ○
        }
    }
}
```

- **尺寸自适应**：通过 `OFFSET_X + (c * CELL_WIDTH)` 公式，将棋盘逻辑坐标 `(r, c)` 精确转换为终端像素位置，实现了 UI 的尺寸自适应。
- **背景颜色设计**：背景色交替设计美观，有助于用户在复杂的棋局中快速定位格点。
- **视觉纯净**：先清空旧格点，再叠放棋子符号，保证了画面的纯净度。

2. 状态反馈

```
if (statusMsg != null && !statusMsg.isEmpty()) {
    // 游戏结束显绿色，报错显红色，普通提示显白色
    graphics.setForegroundColor(gameOver ? TextColor.ANSI.GREEN : (isError ? TextColor.ANSI.RED : TextColor.ANSI.WHITE));

    // 符号增强：利用特定符号（★, ✗, ℹ）辅助颜色进行状态区分
    graphics.putString(OFFSET_X, footerY + 4, (gameOver ? "★ " : (isError ? "✗ ERROR: " : "ℹ ")) + statusMsg);
}
// 实时光标：跟随 inputBuffer 长度动态移动光标，提供真实的输入体验
screen.setCursorPosition(new TerminalPosition(OFFSET_X + prompt.length() + input.length(), footerY + 2));
```

- **语义化提示**：通过“图标+颜色”的双重反馈，极大地降低了命令行界面的学习成本。

2.6.3 设计特点

1. **高内聚低耦合**：`Game` 类只需要传入 `Board` 对象和几个状态字符串，无需了解任何 Lanterna 的坐标计算或颜色设置。
2. **视觉效果优化**：通过 `screen.clear()` 和 `screen.refresh()` 的配合，在每一帧循环中重绘整个界面。虽然是字符界面，但通过高效的 TextGraphics 操作，实现了流畅、无闪烁的视觉效果。
3. **布局适应性**：通过 `CELL_WIDTH`，`CELL_HEIGHT` 等常量控制棋盘比例，方便未来快速调整格点大小以适配不同分辨率的显示器。

3. 开发中的挑战与解决方案

3.1 终端渲染问题

3.1.1 问题描述

在绘制棋盘网格后，左侧的行号（1，2，3...）意外地带有了棋盘末尾格子的背景颜色（蓝色或深灰色），导致界面视觉不统一，文字辨识度降低（见运行截图）。

3.1.2 根源分析

Lanterna 的 TextGraphics 采用的是状态机渲染模式。在内层循环绘制棋盘格子时，调用了 setBackgroundColor()。绘制流程是线性的，当外层循环进入下一行绘制行号时，TextGraphics 依然保留着上一行最后一个格子的背景颜色状态。

3.1.3 解决方案

在 drawGame 方法的外层循环起始处，显式调用 graphics.setBackgroundColor(TextColor.ANSI.DEFAULT)。
效果：强制重置每行起始点的渲染状态，确保装饰性文字（行号、状态栏）处于干净的默认背景下。

3.2 等宽字体下的视觉比例修正

3.2.1 问题描述

由于终端字符通常是矩形（高度约为宽度的 2 倍），简单的 8×8 打印会导致棋盘在视觉上被严重拉长，看起来像一个细长的长方形而非正方形。

3.2.2 根源分析

终端单元格（Cell）的宽高比不平衡。

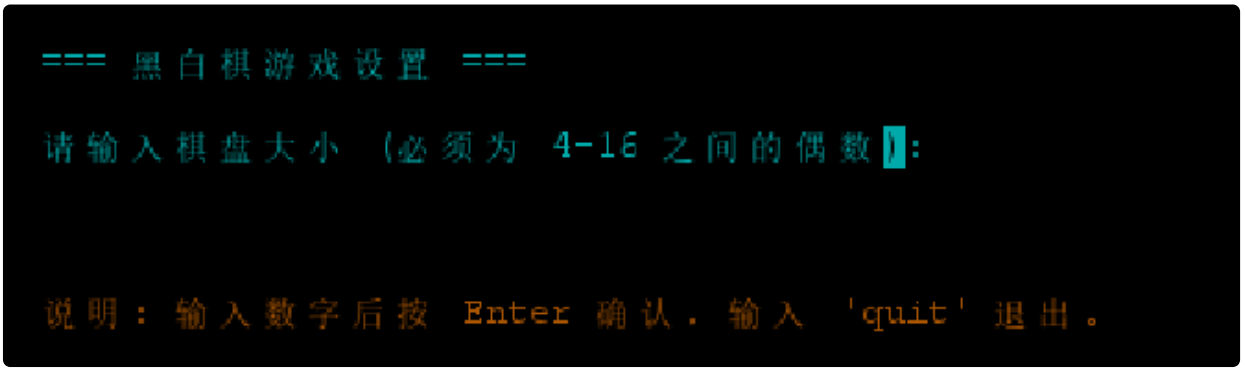
3.2.3 解决方案

在 TerminalUI 中定义常量 CELL_WIDTH = 5 和 CELL_HEIGHT = 3。

- 逻辑实现：在绘制每一个棋盘格子时，通过双重循环填充一个 5×3 的色块，并在坐标 (2, 1) 的位置居中绘制棋子符号。
- 效果：通过数学比例补偿，在物理屏幕上模拟出了接近正方形视觉比例的棋盘网格，极大地提升了用户的视觉体验。

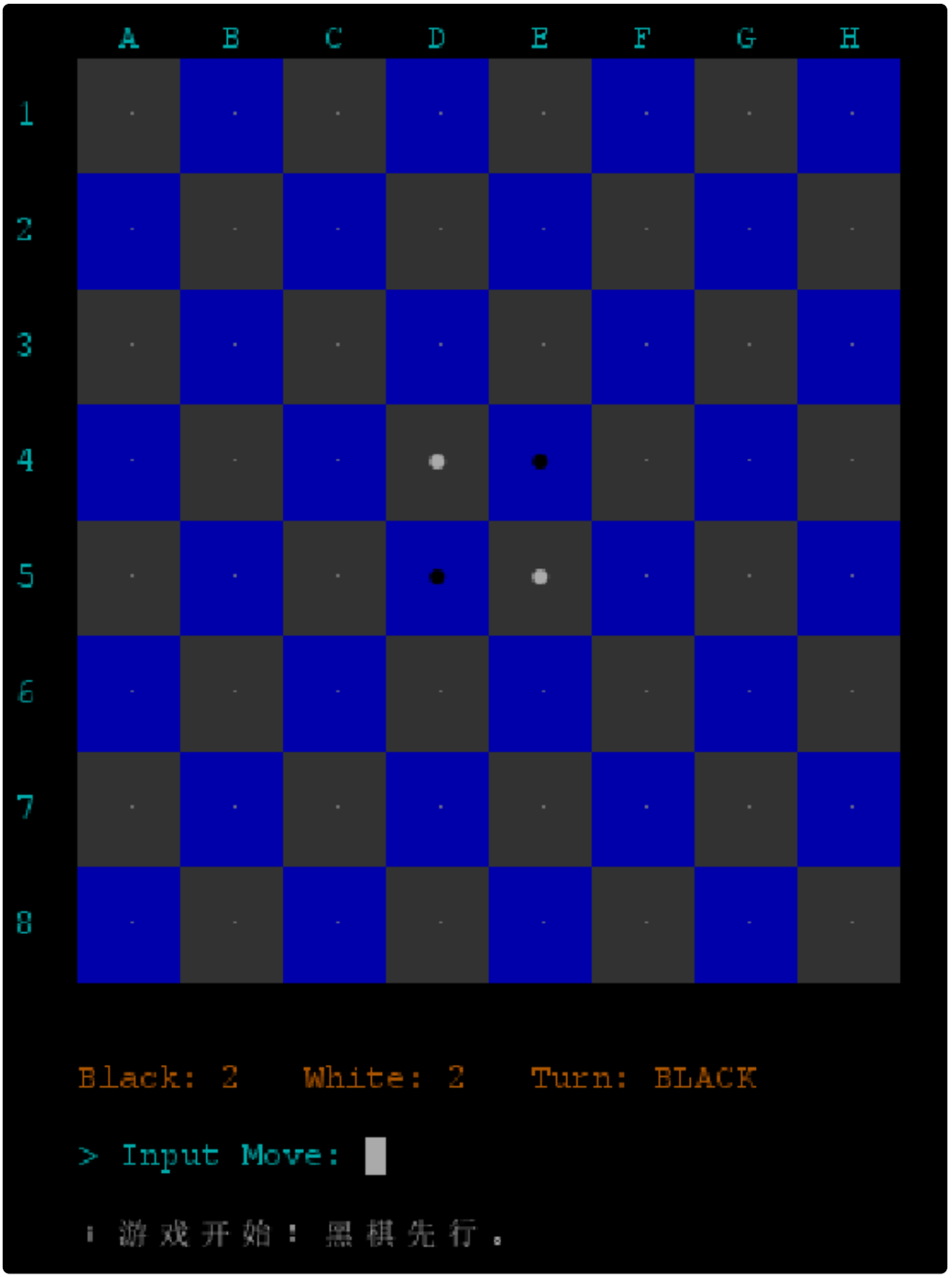
4. 运行截图及简单说明

4.1 起始界面

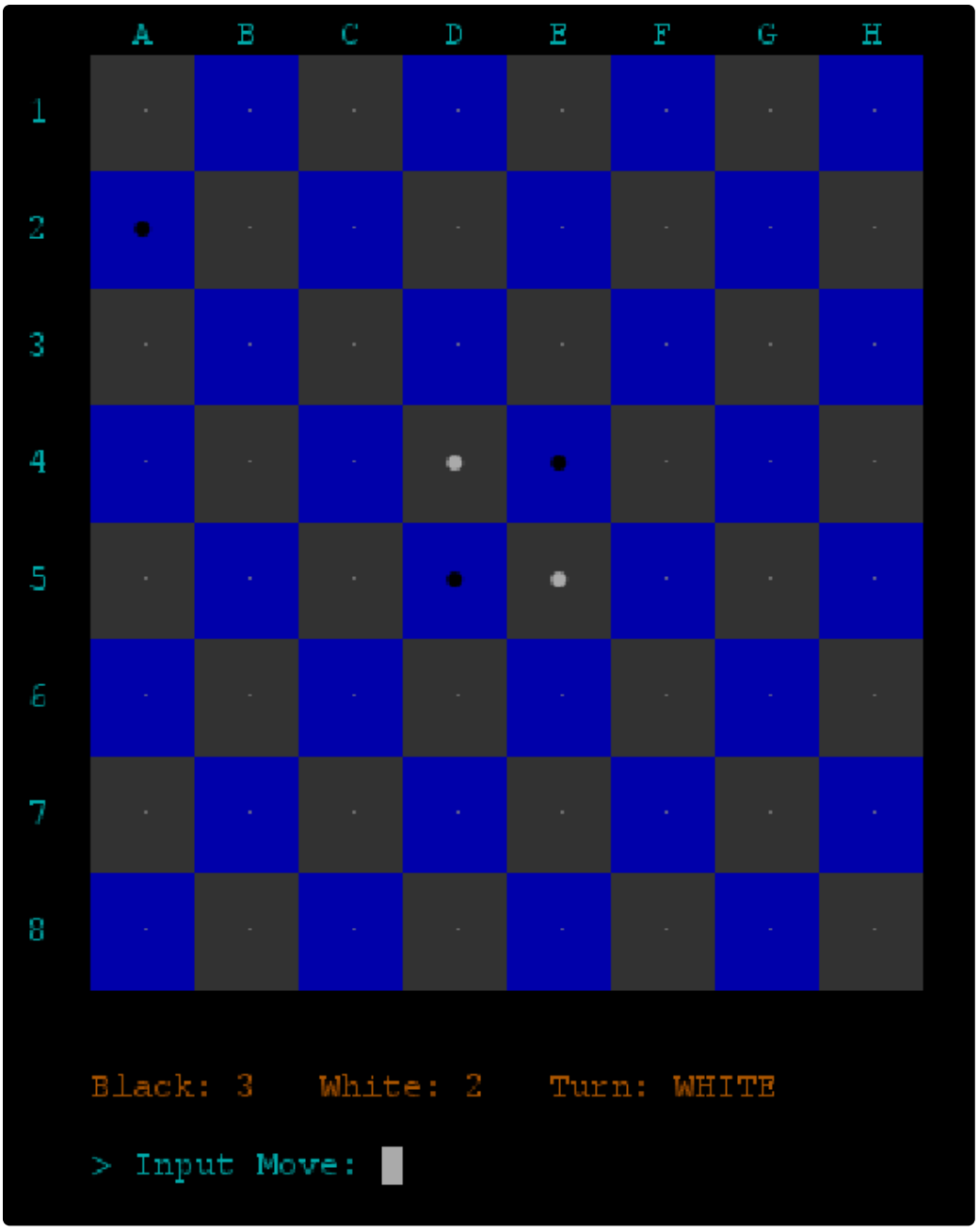


4.2 对战界面（以8x8为例）

1. 初始化界面

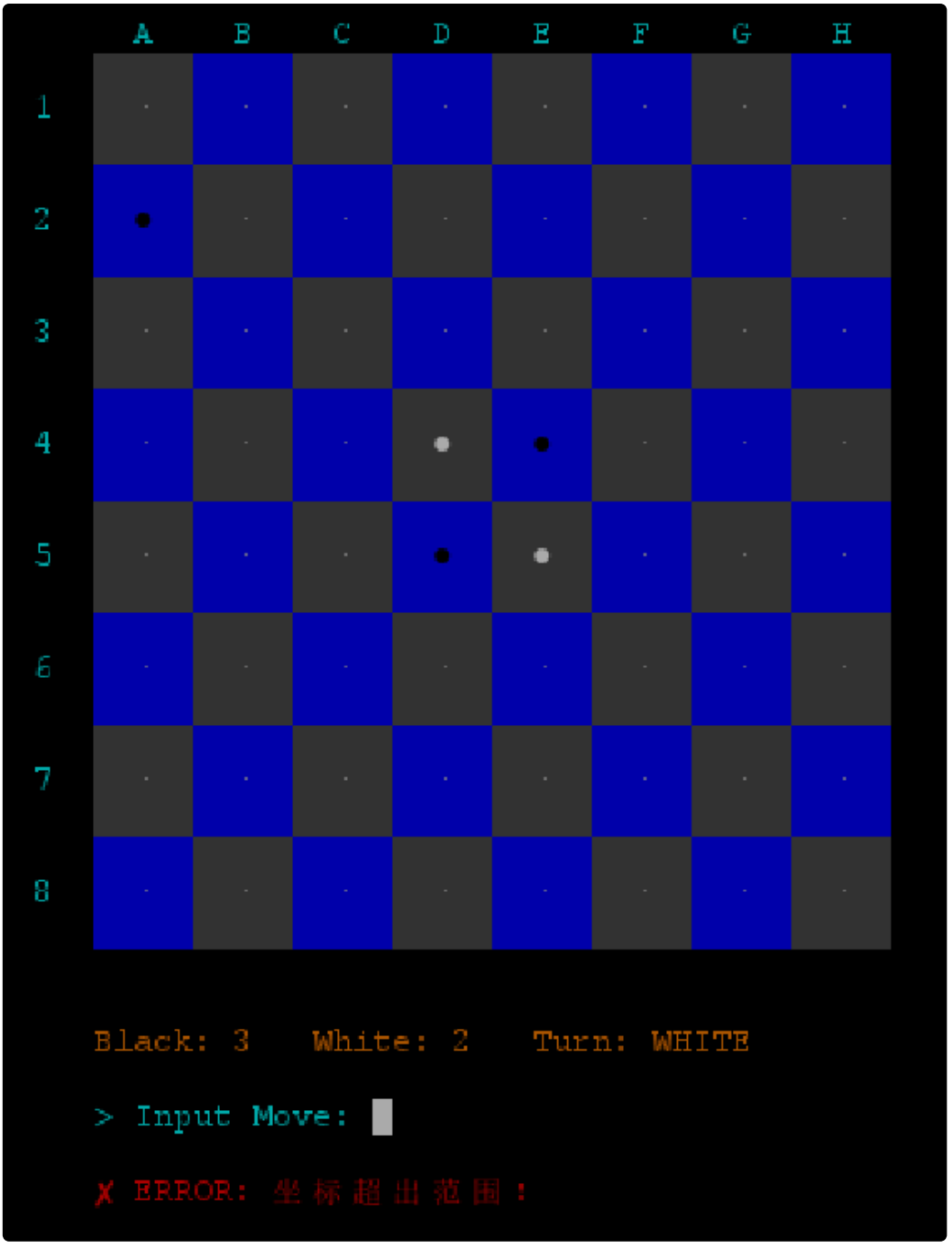


4.3 正常输入界面（输入A2）

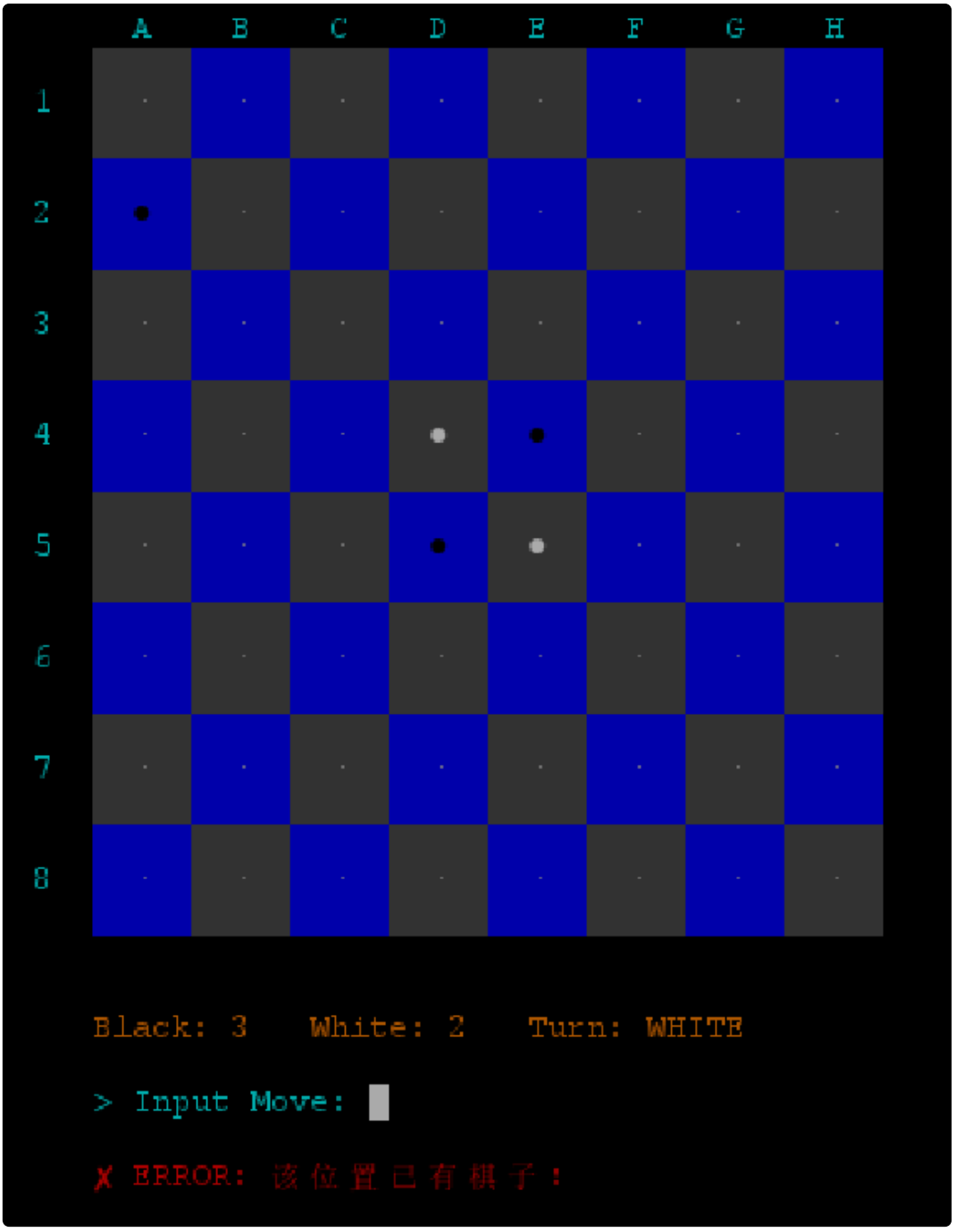


4.4 错误处理界面

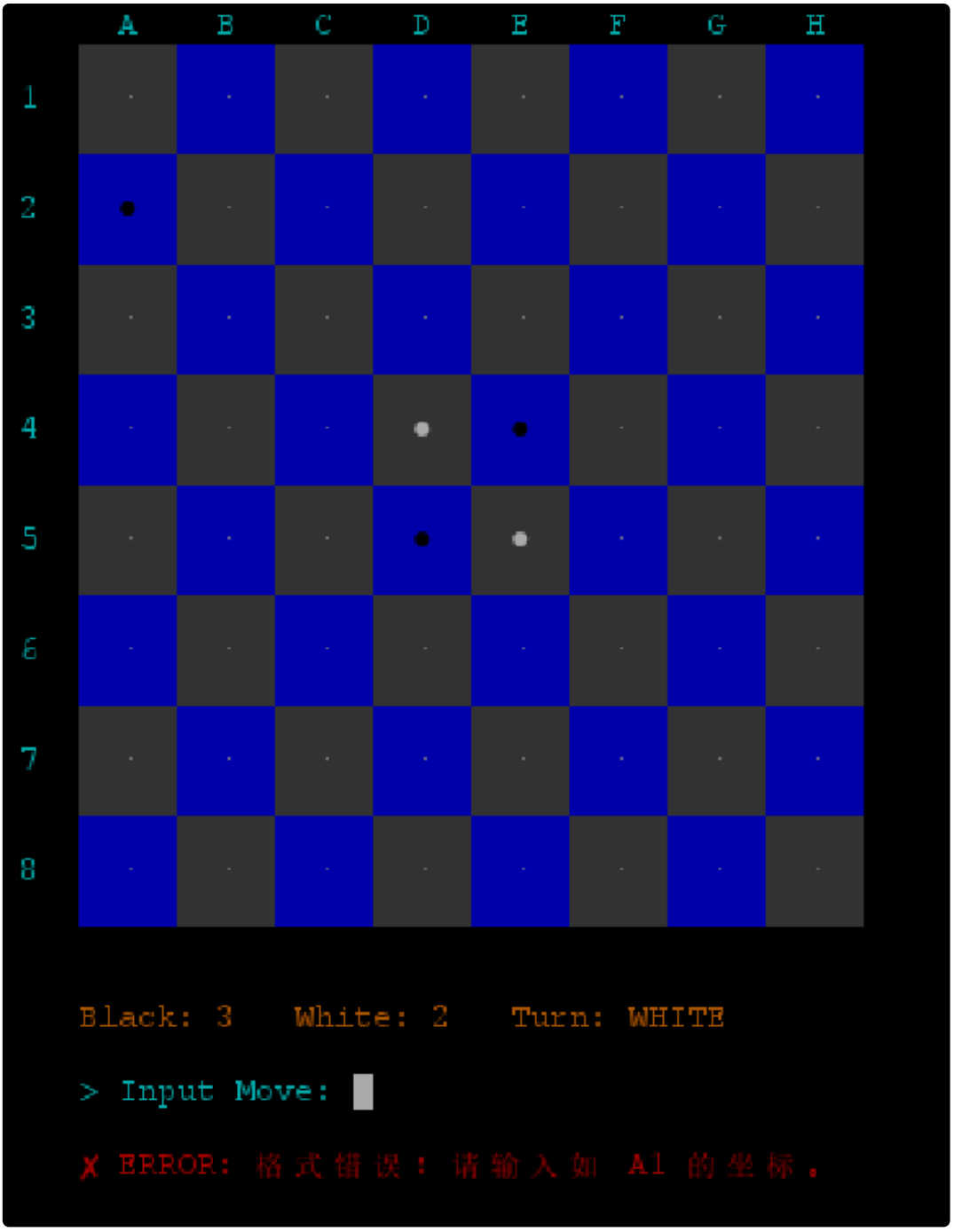
1. 越界（输入A9）


A B C D E F G H
1
2
3
4
5
6
7
8
Black: 3 White: 2 Turn: WHITE
> Input Move:
✗ ERROR: 坐标超出范围！

2. 位置已被占用（输入D4）



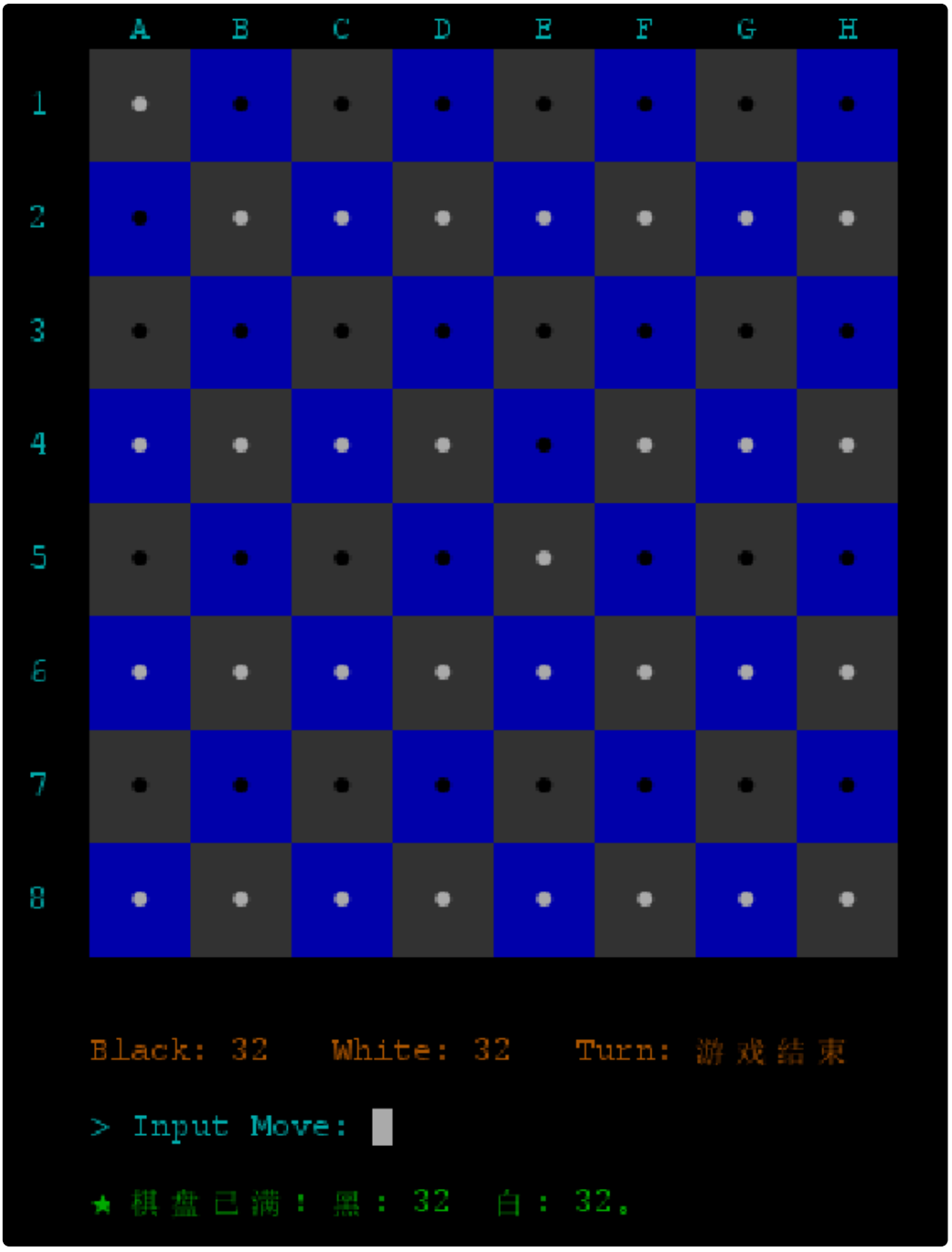
3. 错误输入（输入AA、A$...）



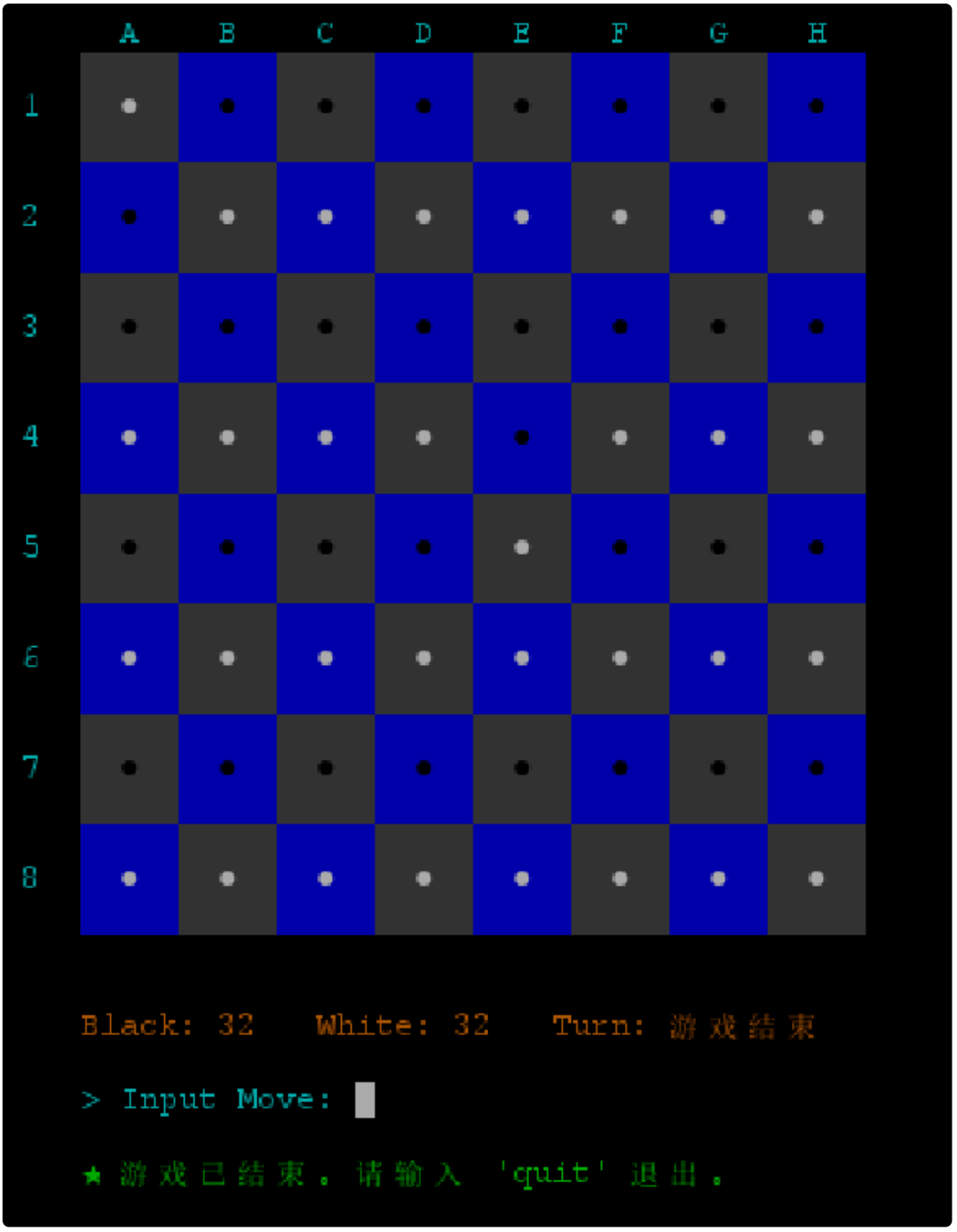
4.5 正常退出界面

4.5.1 截图

1. 棋盘已满


A B C D E F G H
1
2
3
4
5
6
7
8
Black: 32 White: 32 Turn: 游戏结束
> Input Move:
★ 棋盘已满：黑：32 白：32。

2. 游戏结束



5. 可拓展性说明

5.1 游戏规则的进一步实现

目前游戏规则较为简单，对于游戏规则的判定被集成在 `Game` 类中，根据YAGNI原则，还不需要把游戏规则单独成类。在未来的lab中还可能要求使用更复杂的规则，可以新增一个 `GameRules` 类，对游戏规则进行进一步的细化。

5.1.1 吃子反转

- 翻转算法实现：需要引入八向扫描算法。当玩家在坐标 `(r, c)` 落子时，程序应向米字型的八个方向探测，寻找被对手棋子包围的区间。
- 合法性判定校验：黑白棋规定只有“能发生翻转”的格子才是合法落子点。因此，需要增加一个 `isValidMove(r, c, piece)` 方法。在 `Game` 类调用 `handleMove` 时，先询问 `Board` 该步是否合法，而非单纯检查格子是否为空。

5.1.2 回合切换、结束机制

- **自动跳过逻辑**：在黑白棋中，若当前玩家无处可落子，必须跳过回合。你应该在 handleMove 之后增加一个逻辑：使用方法检查对手是否有合法落子点；若无，再检查自己是否有落子点；若双方都无处可落子，即使棋盘未满也应强制触发 gameOver。
- **胜负权重判定**：目前仅判断棋盘是否已满。发展性改动应在 checkGameOver 中加入对双方棋子数量的动态对比，并判定由于“双方均无棋可走”导致的提前结束。

5.2 游戏可玩性的升级

5.2.1 智能辅助与决策

- **AI 集成**：可以设计一个 AIStrategy 接口，在 Game 类中，如果当前回合属于 AI，则不再等待 readInput，而是直接从 AI 策略类获取 Move 对象并执行。
- **合法位置提示**：为了提升用户体验，可以计算出当前所有合法落子点的集合，并交由 TerminalUI 用特殊符号（如淡淡的 *）在棋盘上标出。

5.2.2 存档与悔棋

- **存档与读档**：由于 Board 和 Piece 都是纯粹的数据模型，可以轻易地将当前棋盘状态序列化为 JSON 或文本文件，实现 save 和 load 命令。
- **悔棋功能**：使用哈希表记录每一轮状态，并保留两轮，可以实现悔棋操作。代码的发展性体现在：由于 Board 数据结构清晰，只需备份二维数组即可轻松回溯。

6. 对于 AI 生成代码的优缺点分析

6.1 优点

1. **架构设计标准**
   AI 能够极快地搭建出符合工程规范的 MVC 架构。例如本项目中，将逻辑、显示和控制彻底分离。这种分层思想对于刚刚接触面向对象思想的新手来说，如果自行设计可能需要查阅大量资料，而 AI 能够快速提供一个可使用的框架。
2. **效率高，语法错误少**
   处理第三方库 Lanterna 的初始化、颜色设置、ASCII 码转换等繁琐且易错的细节时，AI 表现极其出色。它能避免低级的语法错误，并提供规范的 JavaDoc 注释，极大提升了开发初期的效率。
3. **鲁棒性高**
   代码中习惯性包括对各种可能出现错误的检查，如本 Lab 代码中的 `isOutOfBounds` 、`try-catch` 块，可以规避大多数运行时崩溃。

6.2 缺点

1. **忽略上下文**
   本 Lab 中我在处理一个显示错误时，AI在修改代码时，经常不能正确使用其他类中的方法，只专注于我向它提出问题的地方，忽略了这是一个完整的项目，造成修改后产成了其他错误的问题。AI往往只关注于局部的代码实现，而缺乏对特定业务的理解。
2. **算法的平庸性**
   在遍历二维数组时，AI采用了最简单的双重循环遍历，虽然对于小规模数据量，这造成的影响微乎其微。但面对大规模数据时，可能会造成一定的性能影响。

3. **对硬编码的偏好**
   例如在编码终端窗口大小时，AI倾向于给出一个固定的数值来确保当前运行成功，而不是去主动适配多变的环境，可能会在后续开发中埋下隐患。
4. **语义重复**
   出于对安全性的追求，AI会在鲁棒性方面表现过激，如开发过程中AI生成的代码多次检查输入、落子的正确性，这其实是没有必要的，反而会造成资源的浪费

7. 总结与反思

通过本项目可以得出结论AI 是一个极其优秀的辅助，但想让其作为领导者有些困难，今后如果有需要使用AI的开发任务，我倾向于这样使用：

- **AI 负责“面”**：快速铺设架构、处理底层 API 调用、生成规范文档。
- **人负责“点”**：核心算法的设计、复杂状态的追踪、以及细节的打磨。